Lab 14 — ConfigMaps en Secrets

Theorie — hoe dezelfde image alle omgevingen in een cluster bedient, waar je een wachtwoord zet als `-e` en `--env-file` niet meer bestaan, en waarom "base64" nog nooit iemand beschermd heeft.

Doelstellingen

- Het principe van lab 08 — één image, ingespoten configuratie — hernemen met de Kubernetes-objecten: **ConfigMap** en **Secret**.
- Configuratie inspuiten als **omgevingsvariabelen** of als **gemounte bestanden**, en weten wanneer je het ene of het andere kiest.
- Begrijpen wat een Secret echt beschermt, en wat niet.
- Weten wat er gebeurt als je een ConfigMap wijzigt: variabelen bevroren, bestanden bijgewerkt.
- Dat alles koppelen aan de Spring Boot-profielen en de secretsbeheerders van het bedrijf.

---

1. Het probleem van lab 08, op clusterschaal

In lab 08 was de regel: een onveranderlijke image, configuratie die van buiten komt, secrets in bestanden eerder dan in variabelen. De mechanismen waren `ENV`, `-e`, `--env-file`, `--secret`. In het cluster is er geen commandoregel meer: alles is een manifest. En een Pod-template (lab 12) is **hetzelfde voor alle replica's en voor alle omgevingen** — er `APP_PROFILE: prod` vast in zetten zou neerkomen op één image per omgeving herbouwen.

Kubernetes scheidt de configuratie dus van het template in twee objecten: de **ConfigMap** voor wat niet gevoelig is (URL's, profielen, configuratiebestanden), het **Secret** voor wat toegang geeft (wachtwoorden, tokens, certificaten). Het template **verwijst** ernaar op naam; elke omgeving heeft de zijne, en het Deployment verandert niet van namespace tot namespace.

> **→ DOCKER-HERHALING**
> ConfigMap = `--env-file` en de configuratiebestanden gemount als bind mount; Secret = `podman secret` en de conventie `_FILE`. Het verschil: in het cluster worden die objecten bewaard in de API server, geversioneerd met de manifesten, en zijn ze beschikbaar op elke node waar de Pod terechtkomt — geen `hostPath` meer naar een bestand dat alleen op jouw machine bestaat (lab 11, vraag 13).

2. De ConfigMap: sleutels, waarden, bestanden

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
data:
  APP_MESSAGE: "bonjour depuis une ConfigMap"
  APP_PROFILE: "staging"
```

Een ConfigMap is een woordenboek van strings. Een waarde kan een woord zijn (`staging`) of een volledig bestand (`default.conf` met zijn twintig regels): `kubectl create configmap web-nginx --`

`from-file=default.conf=nginx-proxy.conf` maakt de sleutel `default.conf` aan met de inhoud van het bestand. Ze is pas nuttig wanneer een Pod ernaar **verwijst**, op twee manieren:

Inspuiting	Syntaxis in het template	Wat de container ziet	Wijziging van de ConfigMap
omgevingsvariabelen	`envFrom: [configMapRef: {name: api-config}]` (alle sleutels) of `env: [valueFrom: configMapKeyRef]` (één sleutel)	`APP_PROFILE=staging` in `env`	**niet doorgegeven**: de Pod moet opnieuw aangemaakt worden
volume	`volumes: [configMap: {name: web-nginx}]` + `volumeMounts`	één bestand per sleutel in de gemounte map	**doorgegeven** na ongeveer een minuut, zonder herstart

De keuze volgt lab 08: variabelen voor wat de applicatie bij het opstarten leest (Spring Boot, de JVM), bestanden voor gestructureerde of lange configuratie (`nginx.conf`, `application.yaml`, een certificaat). In het praktijklab is nginx de officiële image `nginx:alpine` **ongewijzigd**, geconfigureerd door twee gemounte ConfigMaps — de image `web:1.0` van lab 13 is niet meer nodig.

VALKUIL
Een omgevingsvariabele wordt bij de aanmaak in het proces gekopieerd: de ConfigMap wijzigen verandert **niets** aan de lopende Pods. `kubectl rollout restart deployment/api` maakt ze opnieuw aan met de nieuwe waarden. Een gemount bestand is een symbolische link naar een map die de kubelet bij zijn periodieke synchronisatie vervangt (`..data/`, zichtbaar in `ls -l`) — behalve als de mount `subPath` gebruikt, dat het bestand één keer voorgoed kopieert. En een gerefereerde sleutel die niet bestaat (`key: profil` in plaats van `APP_PROFILE`) blokkeert de Pod in `CreateContainerConfigError`: `couldn't find key profil in ConfigMap`.

→ SPRING BOOT
De twee wegen combineren vanzelf: een ConfigMap met variabelen (`SPRING_PROFILES_ACTIVE`, `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app`) en een bestands-ConfigMap gemount op `/config/application.yaml`, die Spring Boot zonder extra configuratie leest. Voor de secrets maakt `spring.config.import=optional:configtree:/run/secrets/` van elk gemount bestand een property (`db.password` uit `/run/secrets/db.password`). De voorrang blijft die van lab 08: variabele > extern bestand > bestand in de JAR.

3. Het Secret: hetzelfde, met het etiket "gevoelig"

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:
  POSTGRES_PASSWORD: "S3cr3t-pg"
```

Een Secret schrijf je zoals een ConfigMap (`stringData` voor tekst, `data` voor base64) en spuit je op dezelfde manier in: `env` met `secretKeyRef`, of een volume `secret:` dat één bestand per sleutel mount. Wat het toevoegt: de waarden zijn **base64-gecodeerd** in de API (`UzNjcjN0LXBn` voor `S3cr3t-pg`), de kubelet mount ze in **tmpfs** (nooit op de schijf van de node), de RBAC-rechten (lab 16) kunnen `get secrets` verbieden aan wie ConfigMaps mag lezen, en de tools (`kubectl describe`, dashboards) maskeren ze.

> **→ BEVEILIGING**
> Base64 is **geen** versleuteling: `kubectl get secret db-secret -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d` geeft het wachtwoord aan iedereen die het Secret kan lezen. In etcd staan de Secrets standaard in klare tekst; serieuze clusters schakelen **versleuteling in rust** in (`EncryptionConfiguration`, of een cloud-KMS) en beperken de toegang tot etcd. Een `Secret`-manifest met zijn waarde **gaat nooit in Git**: je zet het er versleuteld in (SOPS, Sealed Secrets) of laat het door een controller genereren vanuit een externe beheerder (External Secrets Operator + Vault, AWS Secrets Manager…). De regel van lab 08 — een secret dat in een ticket zichtbaar is, is gecompromitteerd — geldt ook voor een `kubectl get -o yaml` dat in een chat geplakt wordt.

Secrets hebben types: `Opaque` (vrij), `kubernetes.io/tls` (certificaat + sleutel, gebruikt door de Ingress voor HTTPS), `kubernetes.io/dockerconfigjson` (registry-inloggegevens, gerefereerd door `imagePullSecrets` om een privé-image te halen — wat onze `k3s ctr images import` overbodig maakt). Een Secret kan net als een ConfigMap `immutable: true` gemarkeerd worden: niet meer wijzigbaar, dus geen risico op een toevallige wijziging, en minder belasting voor de API server.

4. Vier manieren om aan te maken

Commando	Gebruik
`kubectl apply -f configmap.yaml`	de normale weg: een geversioneerd bestand
`kubectl create configmap x --from-literal=K=v`	een snelle test
`kubectl create configmap x --from-file=sleutel=bestand`	een volledig bestand onder één sleutel
`kubectl create secret generic x --from-literal=… --dry-run=client -o yaml`	het manifest **produceren** (met de berekende base64) zonder het aan te maken, om het daarna te versleutelen

`--from-env-file=api.env` leest een bestand `SLEUTEL=waarde` van lab 08: de overgang van Compose naar Kubernetes gebeurt zonder de configuratie te herschrijven.

5. Wat verandert tussen omgevingen

Het Deployment verwijst naar `api-config` en `db-secret` op hun **naam**; elke namespace (of elk cluster) heeft zijn eigen objecten met die naam, met zijn eigen waarden. Een image promoveren van acceptatie naar productie (lab 10) komt neer op **hetzelfde** Deployment toepassen in een namespace waar de ConfigMaps en Secrets `prod` zeggen. Kustomize en Helm (lab 17) genereren die varianten; het onderliggende mechanisme blijft dit.

> **K3S**
> Niets specifieks: ConfigMaps en Secrets zitten in de kern van de API, en k3s bewaart de Secrets in klare tekst in SQLite zoals een kaal cluster ze in etcd bewaart. De optie `--secrets-encryption` van k3s schakelt versleuteling in rust in; een beheerd cluster biedt het als vinkje aan. Op je werkstation is dat niet het punt: de database `k3s.db` is van jou.

6. In het bedrijf

- **Eén ConfigMap per applicatie en per omgeving**, geversioneerd met de manifesten; de waarden die verschillen (`SPRING_PROFILES_ACTIVE`, database-URL, logniveaus) leven daar, nooit in het template.
- **Secrets worden gegenereerd, niet geschreven**: External Secrets Operator of een equivalent synchroniseert ze vanuit de secretsbeheerder van het bedrijf; de Git-repository bevat alleen de *verwijzing* ("het wachtwoord van de database `app` in prod").
- **Bestanden voor de secrets** (`volume: secret`, conventie `_FILE` / `configtree`), variabelen voor de configuratie: dezelfde discipline als in lab 08, omdat `kubectl exec … env` de hele omgeving toont.
- **Rotatie**: een Secret gemount als volume wordt bijgewerkt zonder herstart, op voorwaarde dat de applicatie het bestand opnieuw leest; een Secret als variabele vereist een `rollout restart`. Dat is een ontwerpcriterium, geen detail.
- **RBAC**: de ConfigMaps van een namespace lezen is gebruikelijk; zijn Secrets lezen is voorbehouden.

Onthouden

- ConfigMap = niet-gevoelige configuratie, Secret = toegang; beide zijn woordenboeken waarnaar het template op naam verwijst.
- Variabelen (`envFrom`, `valueFrom`) voor wat bij het opstarten gelezen wordt; volumes voor configuratiebestanden en secrets.
- Een ConfigMap wijzigen verandert de variabelen van lopende Pods niet (`rollout restart`); een gemount bestand wordt vanzelf bijgewerkt, behalve met `subPath`.
- Een ontbrekende sleutel blokkeert de Pod: `CreateContainerConfigError`, `couldn't find key`.
- Base64 is geen versleuteling; een Secret bescherm je met RBAC, versleuteling in rust, en door het nooit in klare tekst te committen.
- `--dry-run=client -o yaml` produceert een manifest; `--from-env-file` herneemt een `api.env` van lab 08.
- Hetzelfde Deployment overal, andere ConfigMaps en Secrets per namespace: zo promoveer je een image.

Woordenschat

ConfigMap: woordenboek van niet-gevoelige configuratie. — **Secret**: woordenboek van gevoelige waarden, base64-gecodeerd, gemount in tmpfs. — **`envFrom`** / **`valueFrom`**: inspuiting van alle sleutels / van één sleutel als variabele. — **volume `configMap`** / **`secret`**: inspuiting als bestanden, één per sleutel. — **`subPath`**: mount van één enkel bestand, zonder automatische bijwerking. — **`CreateContainerConfigError`**: gerefereerde sleutel of object niet gevonden. — **`stringData`** / **`data`**: waarden in klare tekst / in base64 in een Secret. — **`immutable`**: niet-

wijzigbaar object. — **versleuteling in rust**: bescherming van de Secrets in etcd. — **External Secrets / Sealed Secrets / SOPS**: tools om nooit een secret in klare tekst te committen.